Signalinvest
Programação Orientada a Objetos

Estrutura, conceitos e justificativas da implementação

Objetivo do documento. Explicar como o projeto Java do Signalinvest foi estruturado nesta nova etapa da atividade avaliativa, mostrando a aplicação de lógica real nos métodos, herança, polimorfismo, encapsulamento, construtores e a classe Main.

1. Estrutura geral do projeto

A estrutura foi mantida simples para facilitar a abertura no IntelliJ e a compreensão da atividade. As classes principais ficam diretamente dentro da pasta main.

```
Signalinvest-Fintech-Heranca
│
├── .idea
├── Signalinvest-Fintech-Heranca.iml
│
└── main
    ├── Main.java
    ├── Usuario.java
    ├── Investidor.java
    ├── Analista.java
    ├── Startup.java
    ├── AvaliacaoStartup.java
    └── Investimento.java
```

Os arquivos .idea e .iml são utilizados pelo IntelliJ. Para a atividade, o conteúdo principal está nas classes Java.

2. Visão das classes e responsabilidades

Classe	Responsabilidade
Usuario	Superclasse abstrata que reúne os dados comuns dos usuários do sistema.
Investidor	Representa o usuário que analisa oportunidades e realiza aportes.
Analista	Representa o usuário responsável por avaliar startups.
Startup	Representa as empresas analisadas e disponíveis para investimento.
AvaliacaoStartup	Registra notas e calcula o score da startup.
Investimento	Representa o aporte realizado por um investidor em uma startup.
Main	Classe de execução usada para instanciar objetos e simular o funcionamento do sistema.

3. Herança

Herança é o mecanismo que permite que uma classe reutilize atributos e comportamentos de outra. No Signalinvest, Usuario foi criado como superclasse porque Investidor e Analista possuem dados em comum, como id, nome e e-mail.

```
Usuario
├── Investidor
└── Analista
```

A declaração public class Investidor extends Usuario indica que Investidor é um tipo de Usuario. Da mesma forma, Analista também herda de Usuario. Isso reduz repetição de código e deixa o modelo mais coerente.

4. Por que Usuario é abstrata

Usuario representa um conceito genérico. No domínio do Signalinvest, o usuário precisa possuir um papel específico, como Investidor ou Analista. Por isso a classe foi definida como abstrata: ela serve como base para as subclasses, mas não precisa ser instanciada diretamente.

5. Construtores e uso de super

Os construtores permitem criar objetos já com seus dados essenciais. Nas subclasses, o comando super(id, nome, email) chama o construtor da superclasse Usuario. Assim, a classe pai inicializa os dados comuns e a classe filha inicializa os dados específicos.

Exemplo conceitual: Usuario inicializa id, nome e e-mail; Investidor completa a criação com perfil de risco, saldo e lista de investimentos.

6. Encapsulamento

Encapsulamento protege o estado interno dos objetos. Os atributos foram declarados como private, impedindo alterações diretas por qualquer parte do programa.

O acesso é controlado por métodos públicos, como getters, setters e métodos de negócio. Isso permite validar valores antes de alterar um atributo.

Exemplo: em vez de permitir que qualquer classe altere diretamente o saldo de um investidor, o próprio objeto controla as regras relacionadas a esse saldo.

7. Polimorfismo

Polimorfismo permite que uma referência de um tipo mais genérico execute comportamentos específicos do objeto real.

No projeto, Usuario define o método exibirPerfil(), e Investidor e Analista fornecem suas próprias implementações. Assim, uma variável do tipo Usuario pode apontar para um Investidor ou um Analista, e o Java executará a versão correta do método.

Isso é possível graças à sobrescrita de métodos, indicada por @Override.

8. Abstração

Embora o foco da atividade seja herança, polimorfismo e encapsulamento, o projeto também aplica abstração. A classe Usuario representa apenas as características essenciais que todos os tipos de usuário compartilham, sem precisar conhecer os detalhes de cada papel específico.

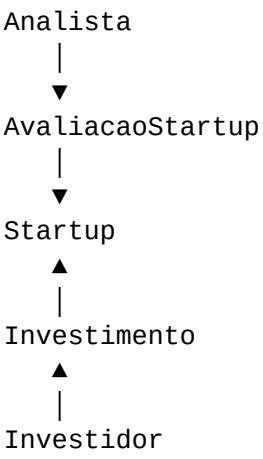
9. Lógica real nos métodos

Na atividade anterior, os métodos podiam apenas exibir mensagens. Nesta etapa, os métodos foram implementados com regras reais do domínio.

O método investir(), por exemplo, verifica se a rodada está aberta, se o valor atende ao investimento mínimo e se o investidor possui saldo suficiente. Se as regras forem atendidas, o saldo é reduzido, um objeto Investimento é criado e a Startup recebe o aporte.

10. Interação entre as classes

As classes não funcionam isoladamente. O Investidor interage com Startup e Investimento; o Analista cria uma AvaliacaoStartup; a avaliação altera o score da Startup. Essa colaboração entre objetos representa melhor o funcionamento de um sistema real.



11. Classe Investidor

Investidor herda os dados comuns de Usuario e adiciona informações próprias, como perfil de risco, saldo disponível e investimentos realizados.

Seu método investir() concentra regras de negócio importantes, impedindo aportes inválidos e coordenando a criação do investimento.

12. Classe Startup

Startup representa as empresas analisadas na plataforma. Entre seus dados estão nome, setor, estágio, valuation, score, rodada aberta, valor mínimo de investimento e valor captado.

Métodos como receberInvestimento() alteram o estado real do objeto, aumentando o valor captado quando um novo aporte acontece.

13. Classe Analista

Analista também herda de Usuario, mas possui características próprias, como especialidade e quantidade de avaliações realizadas. Sua função é produzir avaliações sobre startups.

14. Classe AvaliacaoStartup

AvaliacaoStartup representa a análise de uma empresa. Ela pode armazenar notas financeiras, de mercado, equipe e inovação, calculando um score final. Esse resultado pode ser aplicado à Startup e usado pelo Signalinvest para apoiar decisões de investimento.

15. Classe Investimento

Investimento representa a relação entre um Investidor, uma Startup e o valor aportado. Separar esse conceito em uma classe própria permite que um mesmo investidor possua vários investimentos, cada um com seus próprios dados e cálculos.

16. A classe Main

Main contém o método public static void main(String[] args), ponto inicial do programa. Ela instancia objetos e simula uma execução do sistema.

Na simulação, uma Startup é criada, um Analista realiza a avaliação, o score é atualizado e um Investidor realiza um aporte. Depois, são exibidos resultados como saldo, score, valor captado e projeção do investimento.

17. Fluxo da execução

1. Criar Startup.
2. Criar Investidor.
3. Criar Analista.
4. Abrir a rodada da Startup.
5. Analista avalia a Startup.
6. AvaliacaoStartup calcula o score.
7. Startup recebe o novo score.
8. Investidor solicita um investimento.
9. O sistema valida rodada, valor mínimo e saldo.
10. O objeto Investimento é criado.
11. O saldo do Investidor é reduzido.
12. O valor captado pela Startup é atualizado.
13. São calculadas participação e projeções.
14. A Main exibe os resultados.

18. Evolução em relação à atividade anterior

A atividade anterior demonstrava principalmente a definição de classes, atributos e métodos. Agora, o projeto evolui para uma aplicação orientada a objetos com comportamento.

Antes	Agora

Classes e atributos	Classes, atributos e relações
Métodos com System.out.println	Métodos com lógica real
Pouca interação entre objetos	Objetos colaborando entre si
Sem herança	Usuario como superclasse
Sem polimorfismo	exibirPerfil() sobrescrito
Estrutura básica	Modelo POO completo e coerente

19. Resumo dos conceitos de POO utilizados

Conceito	Aplicação no Signalinvest
Encapsulamento	Atributos private e acesso controlado por métodos.
Herança	Investidor e Analista herdam de Usuario.
Polimorfismo	exibirPerfil() possui comportamento diferente em cada subclasse.
Abstração	Usuario representa genericamente os usuários do sistema.
Construtores	Inicializam objetos com dados coerentes desde a criação.
Interação entre objetos	Investidor, Startup, AvaliacaoStartup e Investimento colaboram no fluxo do sistema.

20. Justificativa da modelagem

A estrutura foi escolhida porque acompanha o domínio real do Signalinvest. O sistema possui diferentes tipos de usuários, startups, avaliações e investimentos. A herança reduz duplicação, o encapsulamento protege os dados, o polimorfismo permite comportamentos específicos e as classes de domínio mantêm responsabilidades separadas.

21. Explicação curta para apresentação oral

No projeto Signalinvest eu criei uma superclasse abstrata chamada Usuario, porque Investidor e Analista possuem dados em comum, como nome, ID e e-mail. Investidor e Analista herdam dessa classe, evitando repetição de código. Para demonstrar polimorfismo, defini o método exibirPerfil() em Usuario e sobrescrevi esse método nas duas subclasses. Assim, uma variável do tipo Usuario pode executar comportamentos diferentes dependendo do objeto real. Os atributos foram definidos como privados para implementar encapsulamento, sendo acessados através de métodos públicos. Também implementei lógica real nos métodos. Ao investir, por exemplo, o sistema verifica se a rodada está aberta, se o valor atende ao mínimo e se o investidor possui saldo. Depois cria um objeto Investimento, reduz o saldo do investidor e aumenta o valor captado pela startup. Por fim, a classe Main instancia os objetos e executa esses métodos para simular o funcionamento do sistema.

22. Conclusão

Esta etapa transforma o Signalinvest de uma simples coleção de classes em um pequeno sistema orientado a objetos. A implementação demonstra os conceitos exigidos pela atividade e, ao mesmo tempo, mantém coerência com a proposta da fintech.